

Subroutines, Calling Conventions, & the Stack

Function linkage, register preservation rules, stack frame management,
and AAPCS64 stack and register rules

Dr. J. Burns

Topic 6 Outline

1. Function Calls and Returns

- Branch-with-link, the link register, returns, leaf functions, and non-leaf functions.

2. AAPCS64 Register Rules

- Argument registers, return registers, caller-saved registers, and callee-saved registers.

3. Stack Frames

- Stack growth, frame records, frame pointer use, prologues, epilogues, and local storage.

4. Worked Function Examples

- Leaf functions, nested calls, recursion, extra arguments, and common linkage mistakes.

PART 1

Subroutine Linkage & Control Flow

High-Level Function Execution Model

- **Subroutines** provide modular abstraction, reusability, and clean software architecture.
- To execute a function call, the CPU must satisfy five core requirements:
 - ① **Pass Arguments:** Transfer input parameters to the function.
 - ② **Transfer Control:** Jump to the function's entry point address.
 - ③ **Isolate Local State:** Provide memory space for function local variables.
 - ④ **Return Value:** Pass computed results back to the caller.
 - ⑤ **Resume Execution:** Return seamlessly to the instruction following the call.

The Link Register (x30) and Branch-with-Link (bl)

- Invoking a function requires remembering where to return when it completes.
- **Branch with Link (bl label):** Atomically performs two operations:
 - ① Saves return address ($PC + 4$) into the **Link Register (x30 / lr)**.
 - ② Branches to target function: $PC \leftarrow \text{label}$.



PC-Relative Calling Range

The `bl` instruction encodes a signed 26-bit immediate offset, allowing direct function calls within a ± 128 MB window of the current instruction.

Subroutine Return Mechanics (**ret**)

- **Return from Subroutine (**ret**):** Branches to the address held in a register. With no explicit operand, **ret** uses the link register: $PC \leftarrow x_{30}$.
- A normal **bl** / **ret** pair therefore resumes at the instruction following the original call.

Example: Minimal Leaf Return Sequence

```
caller_func:
    bl helper_func    // x30 = return address; branch to helper
    mov x1, x0        // resumes here after ret
helper_func:
    add x0, x0, #1     // compute result
    ret               // default target register is x30
```

Leaf vs. Non-Leaf Functions

Leaf Functions

- Call **no other functions**.
- `x30` is not overwritten by a nested bl.
- May still use stack space for locals or spills, but need not save `x30` solely for linkage.

Non-Leaf Functions

- Call one or more **nested subroutines**.
- A nested bl overwrites `x30` with the nested return address.
- Preserve the incoming return address before that happens if it will be needed later.

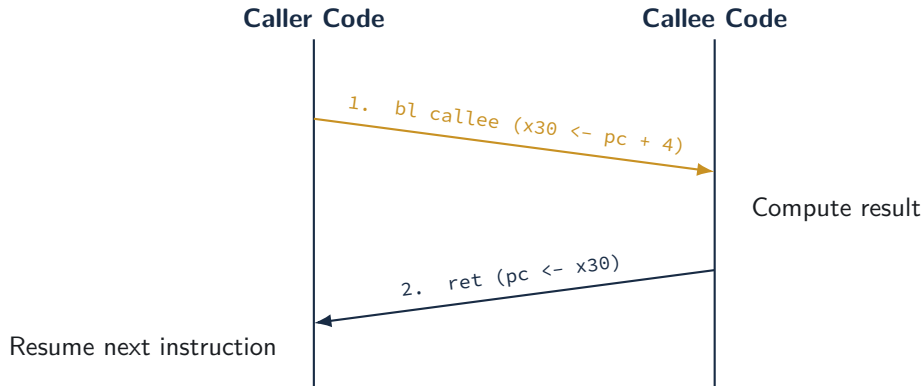
The Non-Leaf Rule

If a function makes another call and must later return to its own caller, its incoming return address must be preserved before `x30` is overwritten.

Indirect Subroutine Calls (b1r)

- When the target function address is determined dynamically at runtime, use **Branch with Link to Register (b1r xn)**.
- Saves return address ($PC + 4$) into x30 and jumps to address held in register xn.
- **Typical uses:** C function pointers and callbacks, C++ virtual method tables, and shared-library veneers.
- **Example:**
 ldr x2, [x0, #8] // load function address
 b1r x2 // call through register x2

Subroutine Call Execution Trace



Control transfers from caller to callee, then returns using x30.

PART 2

Procedure Call Standard & Register Roles

Purpose of Procedure Call Standards (AAPCS64)

- **AAPCS64:** Procedure Call Standard for the ARM 64-bit Architecture.
- Defines the calling-convention contract between independently compiled caller and callee functions.
- **Why the ABI Contract Matters**
 - Allows assembly code to call C library functions such as `printf` and `malloc`.
 - Enables code generated by different compilers to interoperate.
 - Defines parameter passing, return values, stack use, and register-preservation rules.

Parameter Passing Registers (x0–x7)

- Up to **eight 64-bit scalar integer or pointer arguments** can be passed in x0 through x7, subject to AAPCS64 argument-classification rules.

Parameter Index	Register	C Language Example
1st Argument	x0 (w0)	int64_t a
2nd Argument	x1 (w1)	char *str
3rd Argument	x2 (w2)	size_t length
4th–8th Arguments	x3–x7	Additional scalar arguments

Additional Arguments

Arguments not assigned to registers are placed in the stack argument area according to AAPCS64 layout and classification rules.

Return Value Registers

- Most integer and pointer results come back in x0.
- A 32-bit result uses the lower view, w0.
- Some larger register-sized results may use x0 and x1.

Return Data	Register	Example
32-bit integer	w0	int f(void)
64-bit integer or pointer	x0	int64_t f(void)
Two-register result	x0, x1	ABI-dependent small aggregate cases

Floating-point, SIMD, and large-structure return rules are separate cases; this lecture focuses on integer/pointer linkage.

Caller-Saved Registers (x0–x17)

- Also called **scratch** or **volatile** registers.
- The ABI does not require the callee to preserve these registers across a call.

Registers	AAPCS64 Role	Preservation
x0–x7	Arguments and result registers	Caller-saved
x8	Indirect result-location register when required	Caller-saved
x9–x15	Temporary registers	Caller-saved
x16–x17	Intra-procedure-call temporaries (IP0/IP1)	Caller-saved

Caller Responsibility

A caller that needs a value in these registers after a call must preserve that value before executing `bl` or `blr`.

Callee-Saved Registers (x19–x28)

- Also known as **Preserved** or **Non-Volatile** registers.
- Registers x19 through x28 hold long-lived variables across subroutine invocations.

The Callee Contract

If a subroutine modifies any register in x19–x28, its incoming value must be preserved and restored before return.

Typical Preservation (Prologue)

```
str x19, [sp, #-16]!
```

Preserves caller's x19 before modifying it.

Typical Restoration (Epilogue)

```
ldr x19, [sp], #16
```

Restores original x19 before ret.

Special-Purpose ABI Registers

- x29 / fp: frame pointer when a frame record is used.
- x30 / lr: link register; bl and blr write the return address here.
- sp: stack pointer; marks the active stack boundary and must stay properly aligned.
- x18: platform register; do not use it in hand-written examples unless the target platform says it is available.

Course Convention

In hand-written examples for this topic, keep x29, x30, and sp for their linkage and stack roles; use ordinary temporary registers for scratch values.

Register Responsibility Summary

Registers	Role / Usage	Preservation Contract
x0–x7	Arguments and result values	Caller-saved
x8–x17	ABI-specific and temporary uses	Caller-saved
x19–x28	Preserved local values	Callee must restore if modified
x29 (fp)	Frame pointer when used	Callee-saved
x30 (lr)	Return address after bl/blr	A non-leaf callee preserves its incoming return address if needed after another call
sp	Stack pointer	Must be restored to the required value before return and obey alignment rules

PART 3

The Runtime Stack & Stack Frames

The Stack Memory Model

- The runtime stack is a dynamic memory region allocated in process address space.
- **Stack behavior:**
 - Grows **downward** from high memory addresses toward low memory addresses.
 - Stack Pointer (sp) points to the current **lowest active address** (top of stack).
 - Operates on a Last-In, First-Out (LIFO) order.

Allocation vs. Deallocation

- **Allocate:** subtract from sp, e.g., `sub sp, sp, #32`.
- Here #32 means **32 bytes**: room for four 64-bit x register slots.
- **Release:** add the same amount back, e.g., `add sp, sp, #32`.

What Is a Stack Frame?

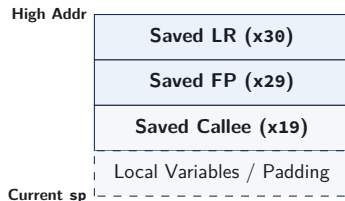
- Each function call may reserve a region of the runtime stack called a **stack frame**.
- A stack frame can hold:
 - a frame record (saved old x29 and saved x30),
 - saved callee-saved registers below it,
 - local variables, temporary storage, and alignment padding.
- The stack pointer (sp) identifies the current stack boundary and changes as stack storage is allocated or released.

Why a Frame Pointer?

When a stable reference within the stack frame is useful, x29 can serve as the frame pointer while sp continues to track the current stack position.

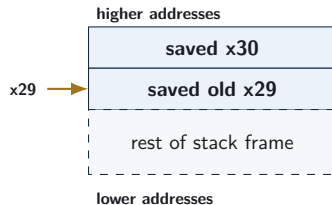
Example Stack Frame Structure

- A **stack frame** stores the stack-resident state associated with one function invocation.
- In the teaching layout used here:
 - ① saved link register (x30) and frame pointer (x29),
 - ② saved callee-saved registers come below it,
 - ③ local variables and buffers extend farther downward.



Frame Pointer (x29) and Frame Records

- When a frame record is used, x29 points to it.
- The record has two 64-bit fields:
 - [x29]: saved previous x29
 - [x29, #8]: saved x30
- The saved previous x29 links this stack frame to the caller's stack frame.



Course Convention

Compilers may omit frame records in optimized code. In this lecture, keeping them visible makes calls, returns, and stack frames easier to trace.

Function Prologue Mechanics

- A prologue saves the frame record first, then grows the rest of the frame downward.
- Here the x29/x30 frame record stays at the high-address end of this function's stack frame.
- Saving one 8-byte x register still reserves 16 bytes so sp remains aligned.

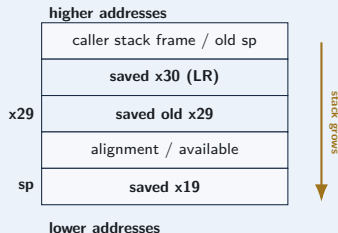
Prologue Template

```
my_func:
    // save old FP and LR as frame record
    stp    x29, x30, [sp, #-16]!

    // x29 anchors the frame record
    mov    x29, sp

    // save x19 below the frame record
    str    x19, [sp, #-16]!
```

Stack Layout



Function Epilogue Mechanics

- The epilogue restores state in the reverse order and releases the stack frame.
- Restore any saved callee registers before restoring the frame record and returning.

Epilogue Template

```
// 1. Restore saved callee register; release its slot
ldr    x19, [sp], #16

// 2. Restore original FP and LR; release frame record
ldp    x29, x30, [sp], #16

// 3. Return to caller
ret
```

The post-indexed `ldr` and `ldp` restore values first, then move `sp` upward. This exactly reverses the prologue's two downward allocations.

Local Variable Allocation on the Stack

C Function with Local Array

```
void process(void) {  
    uint64_t buffer[2];  
    buffer[0] = 10;  
    buffer[1] = 20;  
}
```

Illustrative Leaf Stack Allocation

```
process:  
    sub    sp, sp, #16  
  
    mov    x0, #10  
    str    x0, [sp, #0]  
    mov    x0, #20  
    str    x0, [sp, #8]  
  
    add    sp, sp, #16  
    ret
```

Because this example is a leaf function, it does not need to save x30 solely for return linkage. The 16-byte local allocation preserves stack alignment.

Local Declarations and Stack Layout

C Source Order

```
void f(void) {  
    char a;  
    char b;  
    int  x;  
    char c;  
}
```

- These are independent local variables.
- C source order does not define stack offsets.
- The compiler may reorder, pack, keep in registers, or remove them.

Exception: array elements remain contiguous, and structure members keep declaration order.

One Possible 16-Byte Local Area

Offset	Contents
local + 0	x uses 4 bytes
local + 4	a uses 1 byte
local + 5	b uses 1 byte
local + 6	c uses 1 byte
local + 7–15	padding / available space

The chosen layout must satisfy each object's alignment requirement and preserve the program's observable behavior.

Stack Alignment and Padding Rules

- AAPCS64 requires `sp` to satisfy 16-byte alignment at public interfaces and when used for stack memory accesses.
- Stack-frame allocation therefore rounds the required stack space up to a multiple of 16 bytes.

Calculating Stack-Frame Size

$$\text{Stack Frame Size} = \text{align}_{16}(\text{saved state} + \text{local storage})$$

Required Space	Stack Frame Size	Example Allocation
16B saved FP/LR	16 bytes	<code>stp x29, x30, [sp, #-16]!</code>
16B saved FP/LR + 20B locals	48 bytes	allocate a 48-byte stack frame and place saved state/locals within it
16B saved FP/LR + 32B locals	48 bytes	allocate a 48-byte stack frame

PART 4

Worked Assembly Examples & Patterns

Worked Example 1: Minimal Leaf Function

```
C Code: int64_t add_three(int64_t a, int64_t b, int64_t c)
```

AArch64 Assembly Implementation

```
// Inputs: x0 = a, x1 = b, x2 = c
```

```
add_three:
```

```
    add x0, x0, x1    // x0 = a + b
```

```
    add x0, x0, x2    // x0 = (a + b) + c
```

```
    ret               // result already in x0
```

- Calls no subroutines and needs no stack allocation.
- The function consists of three architectural instructions.

Worked Example 2: Non-Leaf Nested Function

C Code: `int64_t compute(int64_t x)`

```
return add_three(x, 10, 20);
```

AArch64 Assembly Implementation

// Input: x0 = x

`compute:`

```
    stp x29, x30, [sp, #-16]!    // save FP and LR
    mov x29, sp                  // establish frame record
    mov x1, #10                   // 2nd argument
    mov x2, #20                   // 3rd argument
    bl add_three                  // nested call overwrites x30
    ldp x29, x30, [sp], #16       // restore FP/LR
    ret                           // return to caller
```

Worked Example 3: Recursive Factorial

C Logic

```
uint64_t factorial(uint64_t n) {  
    if (n <= 1) return 1;  
    return n * factorial(n - 1);  
}
```

Each invocation preserves its own return address and the current value of *n* across the recursive call.

AArch64 Assembly

```
factorial:  
    cmp    x0, #1  
    b.ls   .Lbase  
  
    stp    x29, x30, [sp, #-32]!  
    mov    x29, sp  
    str    x19, [sp, #16]  
    mov    x19, x0  
    sub    x0, x0, #1  
    bl     factorial  
    mul    x0, x0, x19  
    ldr    x19, [sp, #16]  
    ldp    x29, x30, [sp], #32  
    ret  
  
.Lbase:  
    mov    x0, #1  
    ret
```

Worked Example 4: Passing More Than 8 Arguments

Example Assumption

This example uses ten 64-bit integer arguments. Arguments 1–8 use x0–x7; the remaining arguments are placed in the stack argument area.

Caller Side

```
// Prepare 9th and 10th arguments
mov  x9,  #100
mov  x10, #200
stp  x9, x10, [sp, #-16]!
bl   func_with_10_args
add  sp, sp, #16
```

Callee Side

```
func_with_10_args:
    // Args 1-8 are in x0-x7
    ldr  x9,  [sp, #0]    // 9th arg
    ldr  x10, [sp, #8]    // 10th arg
    ret
```

Common Function Linkage Bugs in Assembly

- **1. Overwriting the Return Address:** A nested `bl` overwrites `x30`; failing to preserve the incoming return address can return control to the wrong location.
- **2. Corrupting Callee-Saved Registers:** Modifying `x19–x28` without restoring them violates the calling convention.
- **3. Stack Misalignment:** Violating AAPCS64 stack-alignment rules breaks ABI assumptions and can fault where SP-alignment checking is enabled.
- **4. Unbalanced Stack Pointer:** Failing to restore `sp` correctly can corrupt stack-resident state and later restores.

SUMMARY & ASSESSMENT

Review and Self-Test Questions

Topic 6: Summary

- **Linkage Primitives:** `bl/blr` write the return address to `x30`; `ret` normally returns through `x30`.
- **AAPCS64 Register Contract:**
 - Scalar arguments commonly begin in `x0–x7`; most scalar results return in `x0`.
 - `x0–x17` are caller-saved; `x19–x28` are callee-saved.
- **Stack Frames:** A stack frame holds saved linkage state, preserved registers, and local storage while maintaining required alignment.
- **Prologue/Epilogue:** Paired `stp/ldp` instructions commonly save and restore `x29/x30` while updating `sp`.

Self-Test Questions: Linkage & AAPCS64

- ❶ **Leaf vs. Non-Leaf Function Linkage:** How do leaf and non-leaf functions differ regarding link-register management and return-address preservation?
- ❷ **Register Preservation Contracts:** Suppose function `foo()` calls `bar()`. If `foo()` needs to maintain a loop counter value across the call, should it store that counter in `x9` or `x19`? Why?
- ❸ **Parameter Passing Limits:** How are integer parameters handled when calling a C function declared with 10 scalar arguments: `void test(int a1, ..., int a10)?`

Self-Test Questions: Stack Frame Management

- ❶ **Stack Allocation Alignment:** A non-leaf function saves x29/x30 (16 bytes) and needs 20 bytes of local buffer space. What is the minimum stack-frame size it must allocate to maintain AAPCS64 compliance?
- ❷ **Prologue/Epilogue Verification:** Identify the bug in the following epilogue:

```
ldp x29, x30, [sp], #16  
ldr x19, [sp, #16]  
ret
```
- ❸ **Frame Pointer Chains:** When frame records are maintained, how does saved x29 help a debugger reconstruct the call chain?

Looking Ahead

Next Lecture Preview

Our Next Topic: Machine Code & AArch64 Instruction Encoding

- **32-Bit Instruction Words:** Relating assembly syntax to fixed-width A64 machine instructions.
- **Encoding Fields:** Identifying opcode, register, immediate, and shift fields within instruction formats.
- **Assembly to Machine Code:** Building and interpreting representative AArch64 instruction encodings.